New idea on temporal learning

Feature semantics drift + relationship drift

✅ 3. Handling Temporal Distribution Shift

This is the major conceptual innovation.

Time-series models and temporal shift

They try to *fit the shift* implicitly through training on sequential data:

- But if feature meaning drifts (e.g., prices after inflation), they fail.
- They do not explicitly model semantic changes.

FMT handles shift explicitly

It assumes feature semantics drift:

- “High heart rate in 2025” ≠ “high heart rate in 2010”
- “Revenue = 1M” means different things depending on economic inflation

FMT uses:

- **temporal embeddings** $\psi(t)$
- to generate **feature-wise transformations** that normalize semantics.

Thus FMT *aligns features across time*, which time-series models normally do not.

PFC inspired

🧠 High-Level Mapping: Brain ↔ ML

Neural Concept (PFC)	Machine Learning Analogue
Short-term working memory	Fast adaptation module
Long-term plasticity	Slow base model learning
Interaction across timescales	Meta-learning / hierarchical updates
Synaptic weight change	Parameter adaptation
Representational drift	Feature + model representation drift

- Models should integrate *fast and slow adaptation mechanisms*
- Relationship drift is best handled when the model itself changes over time
- Feature modulation is not enough — you need parameter dynamics that span multiple timescales, just like the brain's learning system

ML architectures

Arch C — Meta-initialized Fast Learner + Stable Prior (meta-learning style)

Analogy

- Brain has a long-term prior; new tasks adapt quickly from it.

Structure

- Train a base model to be **easy to fine-tune** on new windows (MAML-ish).
- At each new temporal regime, you:
    1. Start from meta-learned init θ_0
    2. Do a few gradient steps on the recent window → θ_t
    3. Use θ_t for predictions on that segment

This makes **fast adaptation itself learned** from past drifts.

Works best if you expect **recurring drift patterns** (e.g., seasonal regimes, repeating economic cycles).

Model update strategy

Think in terms of 3 loops:

1. **Offline slow learning** (long-term plasticity)

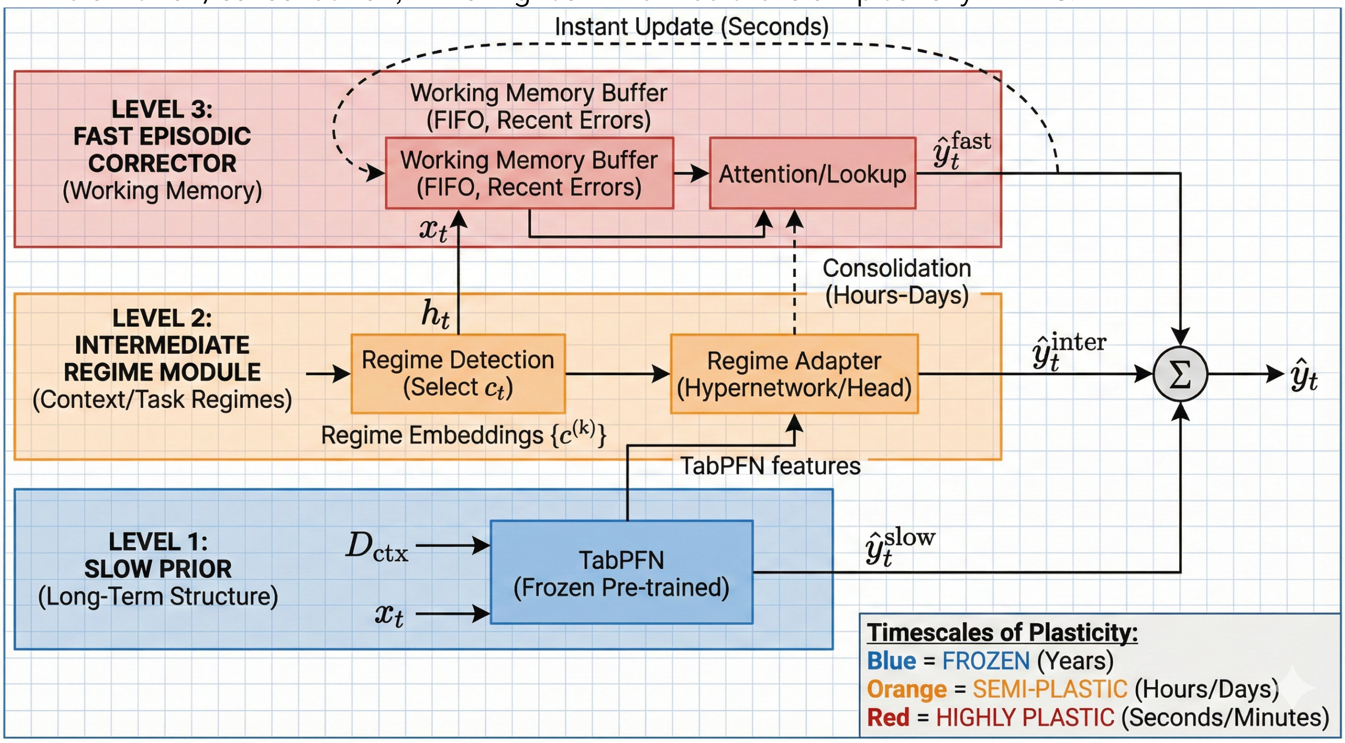
2. **Online fast adaptation** (short-term learning)
3. **Periodic consolidation** (from fast → slow)

TabPFN inspired

> TabPFN provides strong universal inductive bias,
> your system provides temporal, multi-timescale and drift-aware adaptation on top.

1. **Fast–Slow Expert Mixture on top of TabPFN**
    - TabPFN = slow Bayesian prior
    - Small online net = fast drift-aware corrector
    - Drift encoder = router / regularization controller

2. Use TabPFN as a frozen long-term Bayesian prior, wrap it with a slow-updating regime module that captures task- or context-level structure, and add a fast, episodic corrector (parametric or associative) that adapts within short windows; then let information flow upward over time via distillation/consolidation, mirroring fast–intermediate–slow plasticity in PFC.



Timescale 1 — Within an episode (fast; minutes)

- TabPFN and regime module are **fixed.**

- Fast system:
    - continually updates from new data in a small buffer
    - adapts to very local irregularities / micro-drift
    - may be reset or partially decayed when episode finishes

This is your analogue of **working memory / short-term plasticity**.

Timescale 2 — Across episodes within a regime (intermediate; hours–days)

Every so often (e.g., every N episodes or when you see stable pattern in fast corrections):

1. Analyse the **pattern of fast residuals**:
    - If the fast system is consistently compensating in the *same direction* in a region of feature space → suggests true structural relationship drift.
2. **Distil / consolidate into the regime module**:
    - Adjust regime parameters (e.g., adapter weights, regime embedding $c^{(k)}$) to absorb part of what the fast module keeps doing.
    - You can do this via:
    - $\min_{\theta_{\text{inter}}} \mathbb{E}\big[| f_{\text{slow+inter}}(x) - (f_{\text{slow+inter}}(x) + g_{\text{fast}}(x)) |^2 \big]$
    - i.e., make the intermediate module imitate the combination of slow + fast over that window.
3. After consolidation:
    - Slightly **reset or shrink** fast corrections in that area (like sleep-dependent consolidation).

This is analogous to **moving memory from short-term traces to more stable synaptic changes** in PFC.

Timescale 3 — Long-term structural changes (slow; weeks–months)

Much less frequent / more expensive:

- Use large amounts of accumulated data and regime-module parameters to **fine-tune TabPFN** (or to train low-rank adapters inside TabPFN) so that:
  - The prior itself better reflects recurring non-stationarities
  - Over time, TabPFN's internal "task prior" becomes more realistic for your domain

This corresponds to **very slow synaptic reconfiguration** and long-term learning in cortex.
